from the Authorization Server. The Client will have to store them somewhere to be used in subsequent requests from the same user. Tokens are sensitive data that should be secured, and there's no better place than a backend application to do that. But it's not always possible.

Here is my rule of thumb. If the frontend is a mobile or desktop application like iOS or Android, that will be the OAuth2 Client, and it will be categorized as a *public* Client. You can use libraries like AppAuth (https://appauth.io) to add support for OIDC/ OAuth2 and store the tokens as securely as possible on the device. If the frontend is a web application (like in Polar Bookshop), then a backend service should be the Client. In this case, it would be categorized as a *confidential* Client.

The reason for this distinction is that no matter how much you try to hide the OIDC/OAuth2 tokens in the browser (cookies, local storage, session storage), they will always be at risk of being exposed and misused. "From a security perspective, it is virtually impossible to secure tokens in a frontend web application." That's what application security expert Philippe De Ryck writes[a], recommending engineers rely on the backend-for-frontend pattern and have a backend application deal with tokens instead.

I recommend basing the interaction between browser and backend on a session cookie (like you'd do for monoliths) and have the backend application be responsible for controlling the authentication flow and using the tokens issued by the Authorization Server, even in the case of SPAs. That is the current best practice recommended by security experts.

[a] P. De Ryck, "A Critical Analysis of Refresh Token Rotation in Single-page Applications," *Ping Identity* blog, March 18, 2021, http://mng.bz/QWG6

.

Since Edge Service will be the OAuth2 Client in the Polar Bookshop system, let's register it with Keycloak. We can rely on the Keycloak Admin CLI one more time.

Make sure the Keycloak container you started earlier is still running. Then open a Terminal window and enter a bash console inside the Keycloak container:

```
$ docker exec -it polar-keycloak bash
```